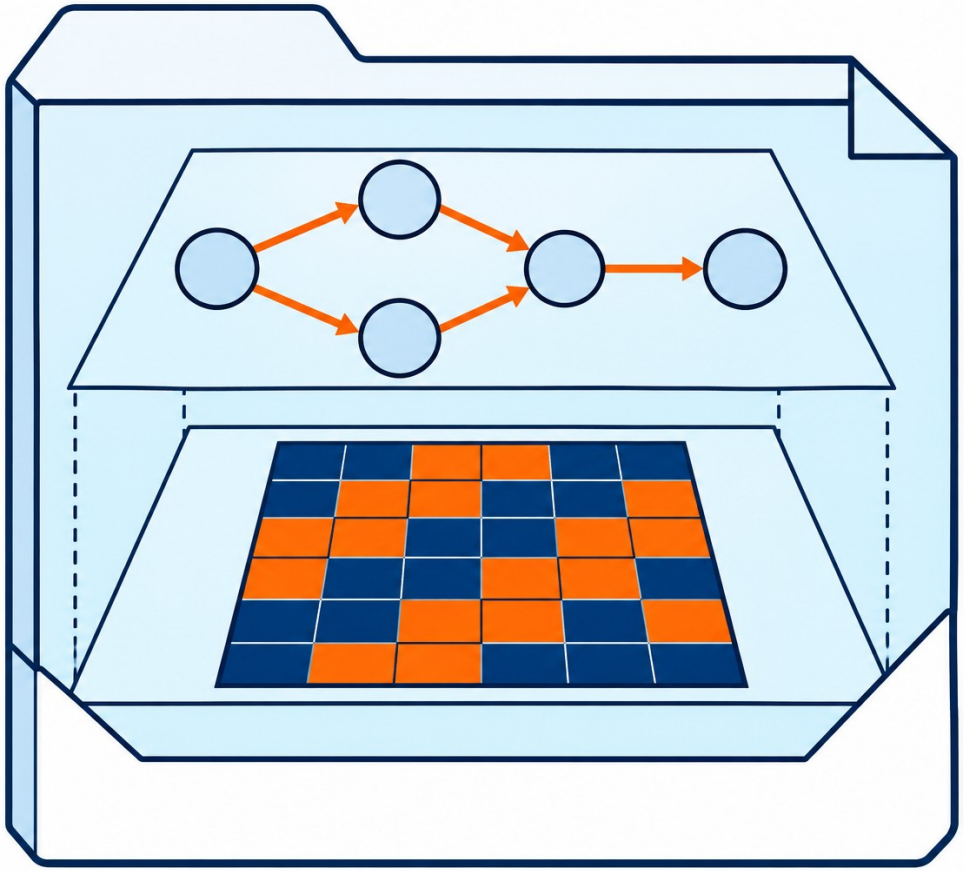
ONNX и TensorRT

Перенос модели и ускорение инференса

Сегодня на занятии

- Что хранит граф ONNX
- Как PyTorch выполняет и экспортирует модель
- Как граф исполняется на CPU и GPU
- Какие решения TensorRT принимает до запуска

Что хранит ONNX



Граф

Операции и связи между ними.

Веса

Значения тензоров, известные до запуска.

Входы и выходы

Имена, типы, размеры.

Подготовка модели в PyTorch

```
model = Classifier()
model.load_state_dict(weights)
model.to(device)
x = x.to(device)
```

state_dict

Веса и буферы; класс модели нужен отдельно.

device

Модель и вход на одном устройстве.

Режимы инференса в PyTorch

Вызов	Что меняется	Когда применять
`model.eval()`	Переключает Dropout и BatchNorm	Проверка и прогноз
`torch.no_grad()`	Отключает запись графа градиентов	Признаки для обучения классификатора
`torch.inference_mode()`	Дополнительно отключает учёт версий	Готовый прогноз

```
model.eval()
with torch.inference_mode():
    y = model(x)
```

Ускорить или экспортировать?

Ускорить повторные вызовы внутри PyTorch

```
fast_model = torch.compile(model)
```

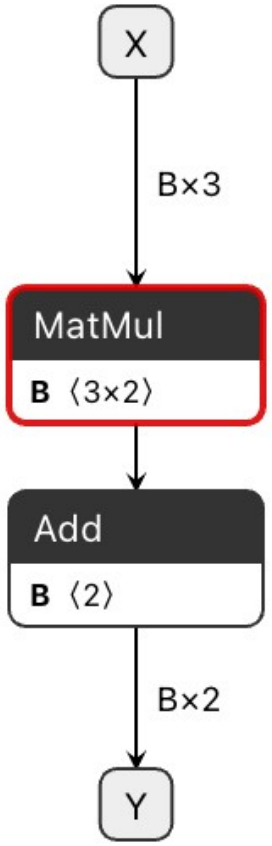
Получить граф с ограничениями на входы

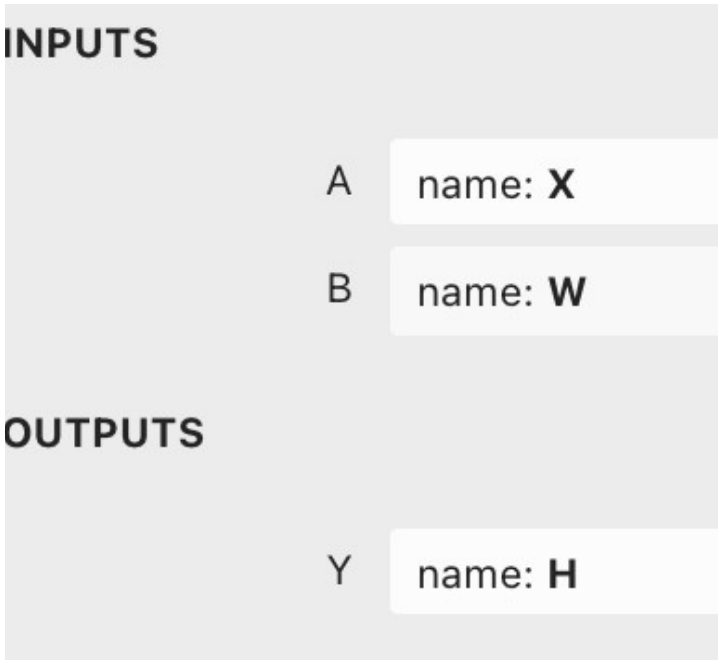
```
ep = torch.export.export(model, (x,))
```

Передать модель в ONNX Runtime или TensorRT

```
torch.onnx.export(model, (x,), "model.onnx", dynamo=True)
```

Netron и GraphSurgeon





Netron

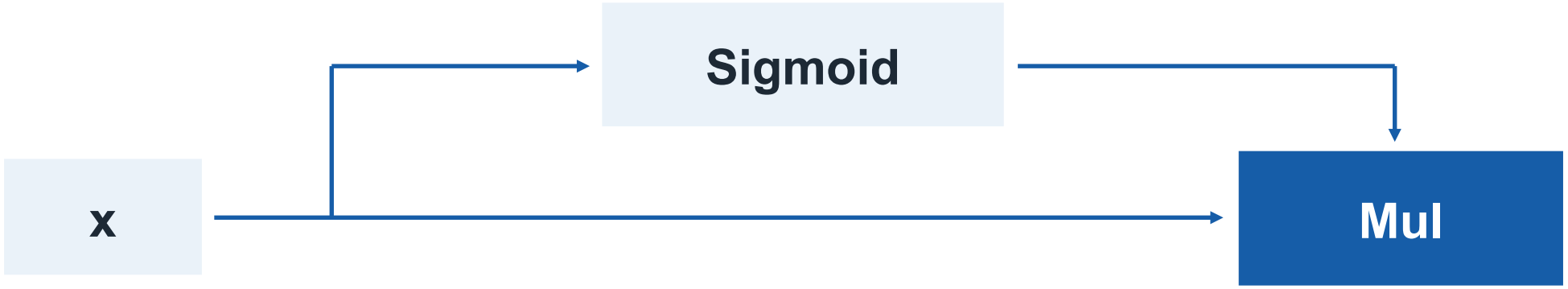
Показывает узлы, входы и веса.

GraphSurgeon

Позволяет заменить узел и переподключить рёбра.

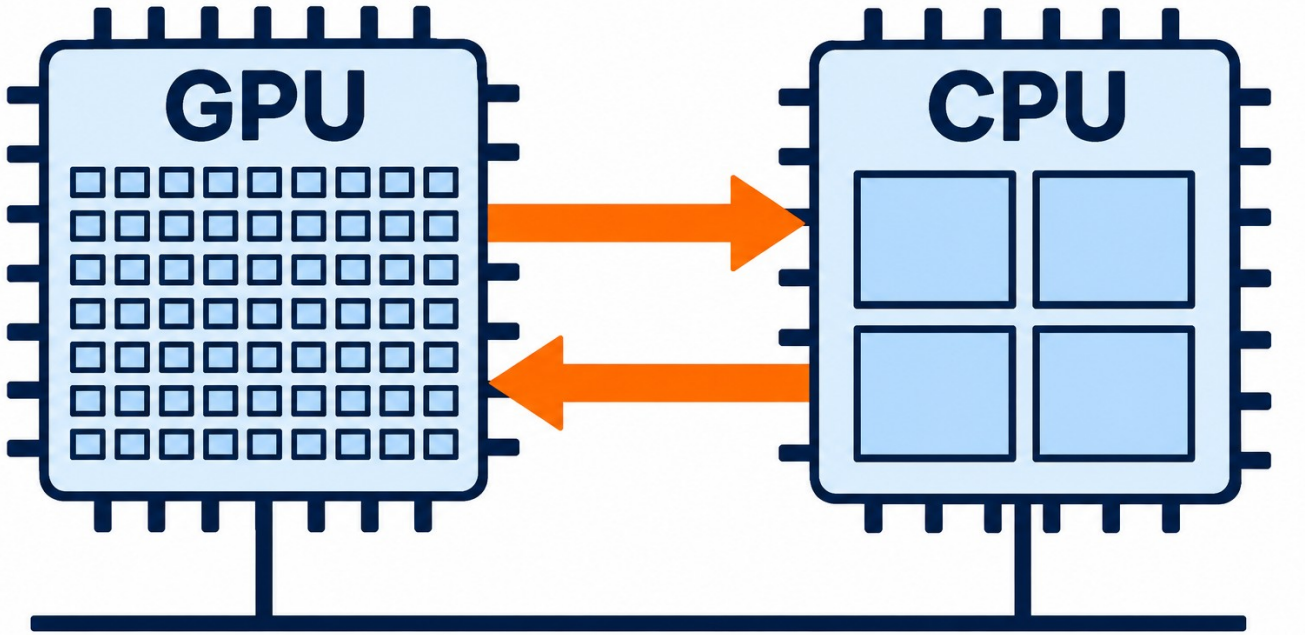
Что делать, если TensorRT не распознал операцию?

$$MySwish(x) = x \cdot sigmoid(x)$$



В GraphSurgeon заменяем узел MySwish двумя узлами: Sigmoid и Mul.

Кто выполняет граф ONNX?



Сессия ONNX Runtime

Хранит граф, готовый к запуску.

Провайдер исполнения

Выполняет поддерживаемые операции на своём устройстве.

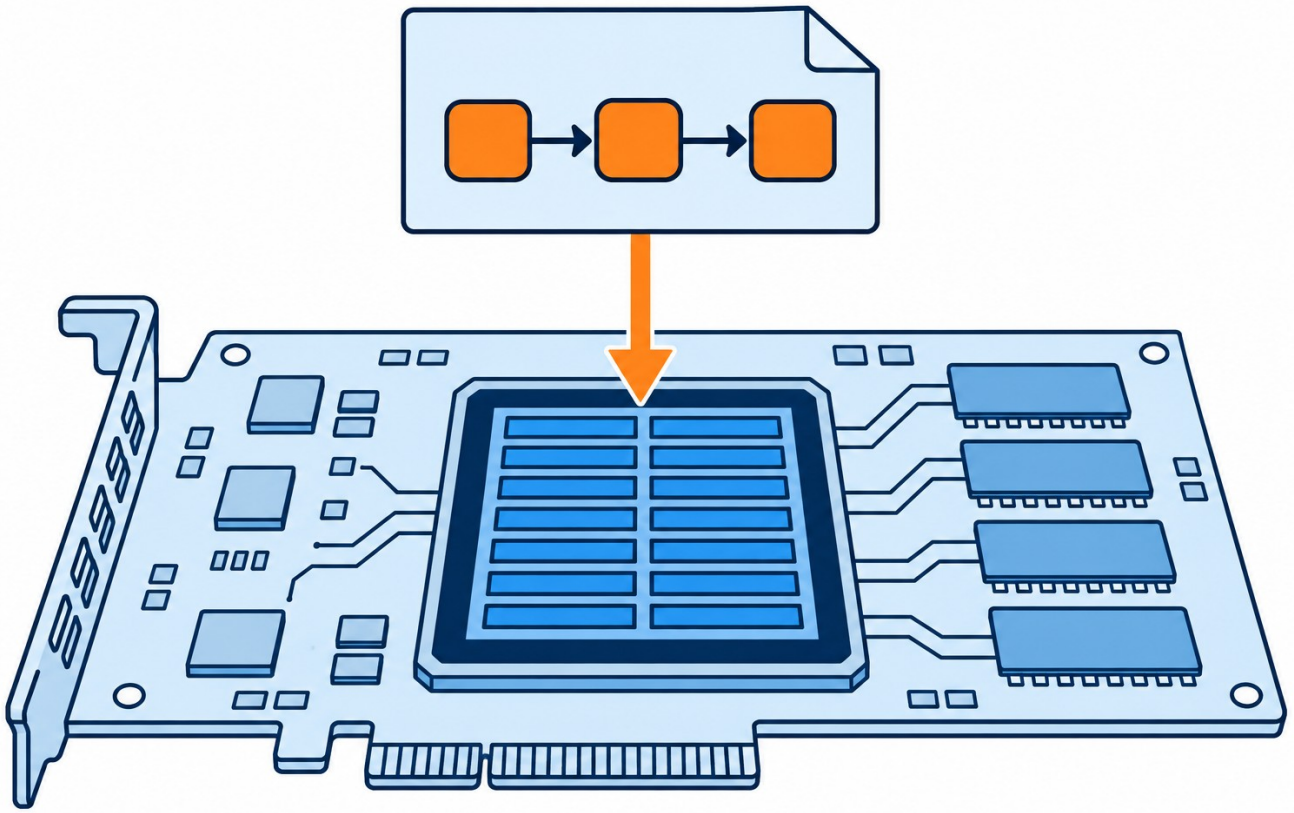
GPU → CPU → GPU: один узел на CPU, две передачи данных.

OpenVINO на CPU



Один раз компилируем модель для процессора, затем подаём новые входы.

Сборка в TensorRT



Парсер

Читает ONNX и строит описание сети.

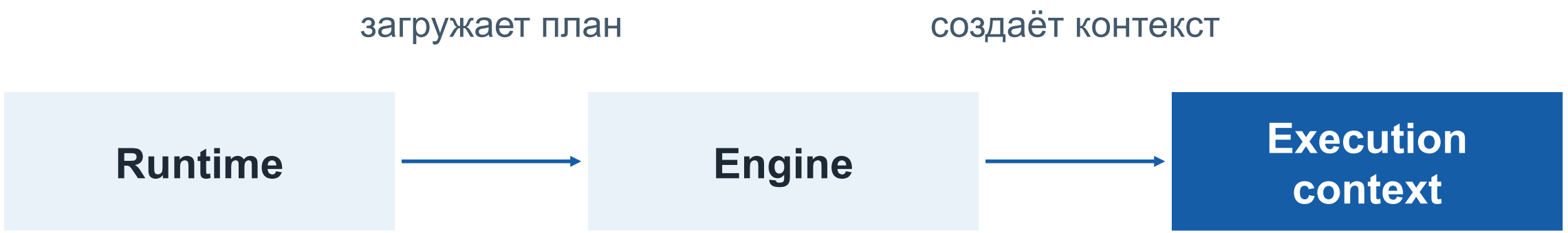
Builder

Выбирает преобразования и реализации слоёв для GPU.

Engine

Готовый план исполнения.

Запуск движка

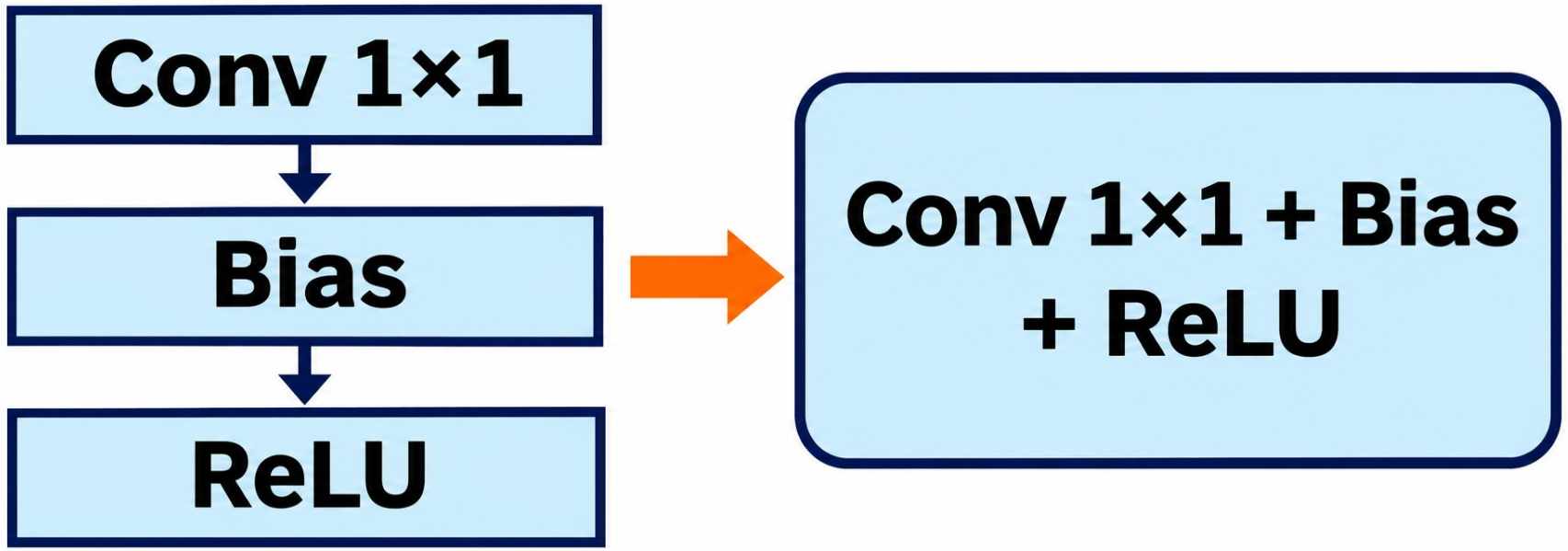


Контекст получает форму входа, адреса буферов и CUDA-поток.

Объединение слоёв

До объединения

После объединения



Свёртка 1×1, смещение и ReLU сливаются в одно ядро. Размер фильтра не меняется.

Формы и выбор ядра



Длина в токенах: min и max задают допустимые входы, opt — размер для подбора тактик.

PTQ и QAT

PTQ: после обучения

По данным подбирают шкалы

Веса уже обучены

QAT: при обучении

При прямом проходе имитируют квантование

Веса подстраиваются под округление

Качество проверяют на отложенной выборке.

Итоги

- ONNX задаёт операции, веса и входы.
- Среда исполнения выбирает, где вычислять граф.
- TensorRT собирает движок под GPU и формы входов.
- При замене модели сравнивают качество и задержку запроса.